



Database Management Systems

Common Table Expressions(CTE)

Anchor:

Dr Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A **Common Table Expression (CTE)** in SQL is a temporary result set that is defined and used within the execution scope of a **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement.
- CTEs are defined using the **WITH** clause and can be referenced multiple times within the main SQL query.

They can be useful for the following:

- Simplifying complex queries
- Breaking down subqueries
- Recursive querying

Basic Syntax of a CTE (with clause–recap)

```
WITH cte_name AS ( SELECT query1.....)  
SELECT query2...FROM cte_name;
```

- **cte_name** is the name of the CTE
- **query1** is a valid SQL query that returns a result set, which will be treated as a virtual table within the main query.
- **query2** is an outer query which uses CTE like a table

Can use **multiple CTEs** separated by commas

```
WITH cte1 AS (...), cte2 AS (...)  
SELECT ...  
FROM cte1 JOIN cte2;
```

Simple example of CTE considering Company Database

```
WITH cte_emp AS (  
    SELECT ssn, fname, dno, salary  
    FROM Employee  
    WHERE salary > 50000  -- filter applied here  
)  
SELECT fname FROM cte_emp; --use of cte
```

Retrieve name, dno, salary of all Employees earning above average salary

```
WITH avg_salary AS (  
    SELECT AVG(salary) AS avg_sal  
    FROM Employee  
)  
SELECT fname, dno, salary  
FROM Employee, avg_salary  
WHERE Employee.salary > avg_salary.avg_sal;
```

Query without using “with”

```
select fname,dno,salary from employee where salary>(select  
avg(salary) from employee);
```

Get employees with salary > 40,000 along with their department name (without CTE)

```
select fname,salary,dname from employee join department on dno=dnumber where salary>40000;
```

Get employees with salary > 40,000 along with their department name using CTE

```
WITH cte_emp AS (  
    SELECT ssn, fname, dno, salary FROM Employee  
    WHERE salary > 40000  
)  
cte_dept AS (  
    SELECT dnumber, dname  
    FROM department  
)  
SELECT e.fname, e.salary, d.dname  
FROM cte_emp e JOIN cte_dept d ON e.dno = d.dnumber;
```

- The WITH clause provides a way of defining a temporary relation whose definition is available only to the query in which the WITH clause occurs.

Example: Find the ssn of the employee who works for the highest number of hours on a particular project. Display the project number and the number of hours the employee has worked as well

Query: WITH max_work(max_hours) AS
(SELECT MAX(Hours) FROM WORKS_ON)
SELECT Essn, Pno, Hours
FROM WORKS_ON, max_work
WHERE Hours = max_hours;

Output:

Essn	Pno	Hours
666884444	3	40.0

- The WITH clause in the query defines the temporary relation *max_work*, which is a single tuple-single attribute relation holding the maximum hours that an employee works. The contents of *max_work* are as shown :

max_hours
40.0

- The relation *max_work* is used in the main query. A join is performed between the *WORKS_ON* and *max_work* relations on the condition that the *Hours* value matches *max_hours*. All tuples in the *WORKS_ON* relation with the maximum value of *Hours* get selected(here there is only one such tuple).
- The *Essn*, *Pno*, and *Hours* values of the selected tuples are retrieved.

- Note that the same result could be achieved using nested queries.

Query: SELECT Essn, Pno, Hours

FROM WORKS_ON

WHERE Hours = (SELECT MAX(Hours) FROM WORKS_ON);

Output:

Essn	Pno	Hours
666884444	3	40.0

- However, using nested subqueries would have made the query harder to read and understand.
- The WITH clause makes the query logic clearer. It also permits this temporary relation to be used in multiple places within a query.

Example: Find all departments where the total salary is greater than the average of the total salary at all departments.

Query: WITH dept_total(Dno,tot_salary)

AS(SELECT Dno, SUM(Salary) FROM EMPLOYEE GROUP BY Dno),

dept_total_avg(avg_tot_salary) AS (SELECT AVG(tot_salary) FROM dept_total)

SELECT Dname, Dnumber FROM dept_total, dept_total_avg, DEPARTMENT

WHERE tot_salary > avg_tot_salary AND Dnumber = Dno;

Output:

Dname	Dnumber
Research	5

```
dept_total(Dno,tot_salary) AS(SELECT Dno, sum(Salary)
FROM EMPLOYEE GROUP BY Dno)
```

- This part of the query retrieves department number and total salary for each department and the result is named *dept_total*. It would look like this:

Dno	tot_salary
1	55000.00
4	93000.00
5	133000.00

```
dept_total_avg(avg_tot_salary) AS (SELECT AVG(tot_salary)FROM dept_total)
```

- This part of the query retrieves the average of total salaries in all departments by making use of *dept_total* which was defined previously. The result is named *dept_total_avg*.

- *dept_total* and *dept_total_avg* are then joined on the condition that *tot_salary* is greater than *avg_tot_salary*.
- The result of the previous join is joined with *DEPARTMENT* to obtain the department name, which is not available in the result of the join between *dept_total* and *dept_total_avg*.

Recursive CTEs

- A recursive **CTE** is one that references itself within that **CTE**.
- The recursive **CTE** is useful when working with hierarchical data as the CTE continues to execute until the query returns the entire hierarchy.
- A typical example of hierarchical data is a table that includes a list of **employees** like in the company database employee has recursive relationship, which represents the employee–supervisor relationship.
- In SQL, recursive queries are typically implemented using Common Table Expressions (CTEs) with the WITH RECURSIVE clause.

Here's the general syntax for creating a recursive query

WITH RECURSIVE cte_name (column_list) AS (

-- Anchor member: The initial query that forms the base case

SELECT ...

FROM ...

WHERE ...

UNION ALL

-- Recursive member: The query that refers to the CTE itself

SELECT ...

FROM cte_name **-- Refer to the CTE within itself**

WHERE ...

)-- The final SELECT statement outside the CTE to retrieve the result

SELECT ...FROM cte_name;

Here's an explanation of each part of the syntax:

- **cte_name:** This is the name you give to the Common Table Expression (CTE), which you can use to refer to it within the CTE itself and in the final SELECT statement.
- **column_list:** Specify the list of columns you want to include in the CTE's result set.
- **Anchor member:** This is the initial query that forms the base case of the recursion. It selects the starting point or points for the recursive process. It's important to ensure that this query doesn't reference the CTE itself.

Here's an explanation of each part of the syntax:

- **UNION ALL:** This is used to combine the results of the anchor member with the results of the recursive member. UNION ALL is used because it includes duplicate rows, which is often necessary in recursive queries.
- **Recursive member:** This is the query that refers to the CTE itself, creating the recursion. It selects records based on relationships established in previous iterations. This query must reference the CTE by its name and should include a termination condition to stop the recursion.
- **Final SELECT statement:** This query retrieves the final result set from the CTE. It typically follows the recursive part of the CTE definition.

Example: Given an example of a recursive query that counts down from 5 to 1:

```
WITH RECURSIVE Countdown (n) AS (
```

```
-- Anchor member
```

```
SELECT 5
```

```
UNION ALL
```

```
-- Recursive member
```

```
SELECT n - 1
```

```
FROM Countdown
```

```
WHERE n > 1
```

```
)
```

```
-- Final SELECT statement
```

```
SELECT n FROM Countdown;
```

n
5
4
3
2
1

Example: Given an example of a recursive query that counts down from 1 to 5:

```
WITH RECURSIVE cte(n) AS (  
    -- Anchor part of the query  
    SELECT 1  
    UNION ALL  
    -- Recursive part of the query  
    SELECT n + 1 FROM cte WHERE n < 5  
)  
SELECT * FROM cte;
```

n
1
2
3
4
5

Execution Flow:

A **recursive CTE** has two parts:

1. **Anchor member** → the initial result set
2. **Recursive member** → repeatedly applied until it stops producing rows

The database engine runs it in an iterative fashion.

Run the anchor query first: `SELECT 1`

This produces the first row: 1.

Then, Apply the recursive part - SQL substitutes the current contents of cte into:

```
SELECT n + 1 FROM cte WHERE n < 5
```

From (1), since $1 < 5$, it generates: 2 which is added to the result.

Give a query to extract the organizational hierarchy beginning with a specific employee, typically the CEO, and extending downward through the reporting structure.

```
with recursive orghierarchy as(  
  select ssn,fname,lname,super_ssn, 1 as level  
  from employee  
  where super_ssn is null  
  union all  
  select e.ssn,e.fname,e.lname,e.super_ssn, OH.level+1  
  from employee e  
  join orghierarchy OH on e.super_ssn=OH.ssn  
)  
select * from orghierarchy;
```

Output:

ssn	fname	lname	super_ssn	level
888665555	James	Borg	NULL	1
123456789	John	Smith	888665555	2
333445555	Franklin	Wong	888665555	2
453453453	Joyce	English	333445555	3
666884444	Ramesh	Narayan	333445555	3
987654321	Jennifer	Wallace	333445555	3
999887777	Alicia	Zelaya	333445555	3
987987987	Ahmed	Jabbar	987654321	4

8 rows in set (0.04 sec)

The **CASE** statement in **SQL** is a versatile conditional expression that enables us to incorporate **conditional logic** directly within our queries. It allows you to return specific results based on certain conditions, enabling dynamic query outputs.

- It is commonly used to create new columns based on conditional logic, provide custom values, or control query outputs based on certain conditions.
- If no condition is true then the ELSE part will be executed. If there is no ELSE part then it returns NULL.

Syntax:

```
CASE  
WHEN condition THEN result1  
WHEN condition THEN result2  
...  
Else result  
END
```

```
CASE expression  
    WHEN value1 THEN result1  
    WHEN value2 THEN result2  
    ...  
    ELSE default_result  
END
```

Write an SQL query to display the first name and salary of each employee, along with a derived column that categorizes employees into 'High', 'Medium', or 'Low' salary levels based on the following conditions:

- Salary $\geq$ 55000 THEN 'High'
- Salary $\geq$ 40,000 THEN 'Medium' else low

Query: SELECT Fname, Salary,

CASE

WHEN Salary $\geq$ 55000 THEN 'High'

WHEN Salary $\geq$ 40000 THEN 'Medium'

ELSE 'Low'

END AS Salary_Level FROM employee;

Fname	Salary	Salary_Level
John	30000.00	Low
Franklin	40000.00	Medium
Joyce	25000.00	Low
Ramesh	38000.00	Low
James	55000.00	High
Jennifer	43000.00	Medium
Ahmed	25000.00	Low
Alicia	25000.00	Low

8 rows in set (0.00 sec)

Write an SQL query to display the first name and salary of each employee, along with a new column called Bonus_Status that categorizes employees based on their salary as follows:

- Salary $\geq 50,000 \rightarrow$ 'Eligible for 20% Bonus'
- Salary between 40,000 and 49,999 $\rightarrow$ 'Eligible for 10% Bonus'
- Salary $< 40,000 \rightarrow$ 'No Bonus'

—TRY this

Expected Output:

Fname	Salary	Bonus_Status
John	30000.00	No Bonus
Franklin	40000.00	Eligible for 10% Bonus
Joyce	25000.00	No Bonus
Ramesh	38000.00	No Bonus
James	55000.00	Eligible for 20% Bonus
Jennifer	43000.00	Eligible for 10% Bonus
Ahmed	25000.00	No Bonus
Alicia	25000.00	No Bonus

Solution for the previous query

```
SELECT
```

```
    Fname, Salary,
```

```
    CASE
```

```
        WHEN Salary >= 50000 THEN 'Eligible for 20% Bonus'
```

```
        WHEN Salary >= 40000 THEN 'Eligible for 10% Bonus'
```

```
        ELSE 'No Bonus'
```

```
    END AS Bonus_Status
```

```
FROM employee;
```

Write a query to display each employee's first name, number of projects they are working on, and a new column Workload_Status as follows:

- 3 or more projects → Overloaded
- 2 projects → Normal Load
- 1 or 0 projects → Light Load

Expected Output:

Fname	Project_Count	Workload_Status
John	2	Normal Load
Franklin	4	Overloaded
Joyce	2	Normal Load
Ramesh	1	Light Load
James	1	Light Load
Jennifer	2	Normal Load
Ahmed	2	Normal Load
Alicia	2	Normal Load

Solution for the previous query

```
SELECT
    e.Fname, COUNT(w.Pno) AS Project_Count,
    CASE
        WHEN COUNT(w.Pno) >= 3 THEN 'Overloaded'
        WHEN COUNT(w.Pno) = 2 THEN 'Normal Load'
        ELSE 'Light Load'
    END AS Workload_Status
FROM employee e
LEFT JOIN works_on w ON e.Ssn = w.Essn
GROUP BY e.Fname;
```

The same query using a CTE to show how CASE can be combined with CTE for better readability

```
WITH Project_Counts AS (  
    SELECT e.Fname, COUNT(w.Pno) AS Proj_Count  
    FROM employee e LEFT JOIN works_on w ON e.Ssn = w.Essn  
    GROUP BY e.Fname  
)  
SELECT Fname, Proj_Count,  
    CASE  
        WHEN Proj_Count >= 3 THEN 'Overloaded'  
        WHEN Proj_Count = 2 THEN 'Normal Load'  
        ELSE 'Light Load'  
    END AS Workload_Status  
FROM Project_Counts;
```

- 1) Which SQL clause is used to define a Common Table Expression (CTE)?
A) CREATE B) WITH C) TEMP D) SET

- 2) A Common Table Expression (CTE) provides a _____ result set that is defined and used strictly within the execution scope of a single SQL statement.

- 3) When creating a recursive CTE, which operator is used to combine the results of the initial base case query with the results of the recursive query?
A) JOIN
B) INTERSECT
C) UNION ALL
D) EXCEPT

4) A recursive CTE consists of two main components: the recursive member, which references the CTE itself, and the _____ member, which forms the initial base case of the recursion.

5) When writing a CASE expression in SQL, what happens if none of the specified WHEN conditions evaluate to true, and no ELSE part is provided?

- A) The query throws a syntax error.
- B) It returns FALSE.
- C) It returns 0.
- D) It returns NULL.

6) If you need to define multiple CTEs for use within a single main query, you can do so by separating each CTE definition with _____.

Answers:

1. B
2. temporary
3. C
4. anchor
5. D
6. commas



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu